PROJECT BRIEF · TRACK 1 · GREENFIELD BUILD

Firewatch

Security alerts for Lightning nodes. It tells an operator, within minutes, whether a published flaw actually affects their node, and exactly what the developers said to do.

> Lightning bugs get fixed in days.
> Nodes get patched in months.
> Firewatch closes the gap.

EVENT	BUILD DAY	DEMO	TEAM
Btrust Bitcoin Build-a-thon DevFest Kaduna 2026	Oct 29, 2026 Stonetech Square, Kaduna	Oct 31, 2026 Conference showcase	3 Rust developers 1 frontend developer

01

Overview

Firewatch is a read-only Rust daemon that connects to an LND or Core Lightning node, matches its version and configuration against a curated advisory database, and sends an encrypted alert quoting the developers' own fix.

TRACK
Track 1, Greenfield Build. All code is written on the build day; this brief, the advisory research and environment setup are prepared beforehand.

TEAM
4 members: 3 Rust developers (including Dave) and 1 frontend developer.

OUTPUT
Three verdicts per advisory:
Affected · Can't tell · Not affected

What it is not: an uptime monitor, an auto-updater, or anything that can change or spend from the node.

02

The problem

In two months, every major Lightning implementation shipped security fixes, and Core Lightning reported attackers going after nodes that had not upgraded.

DATE	SOFTWARE	WHAT HAPPENED
Oct 2, 2026	Core Lightning	Developers urged anyone on 26.06.7 or earlier to upgrade, citing reports of attackers targeting unpatched nodes. Cointelegraph
Sep 22, 2026	Core Lightning	26.06.8 fixed a sender crash, REST memory exhaustion, and a channel-close bug that could lose funds to a penalty. Cointelegraph
Sep 9, 2026	LDK	v0.2.6 fixed splice fee over-allocation to a malicious peer and a bogus-HTLC bug that stops the node from restarting. Release notes
Aug 26, 2026	Core Lightning	Told operators on 26.04 and earlier to take nodes offline before a patch existed. Crypto Briefing
Aug 26, 2026	LND	A separate vulnerability disclosed for versions before 0.21.0. Crypto Briefing

Three gaps let this hurt people

1

Operators hear late

Warnings went out on X and news sites, not to the node itself.

2

"Am I affected?" is hard

Some bugs need a feature on (LDK's splice bug only bites when your node initiates a splice). A version alone gives false alarm or false comfort.

3

The right action changes

Sometimes "upgrade", sometimes "go offline, no fix yet". Operators need the developers' exact instruction, fast.

> **Existing tools miss this.** lightning-sentinel monitors LND status over Telegram but does no advisory matching. Generic release notifiers say a version exists, not whether you are vulnerable.

03

Why it fits this hackathon

Firewatch matches the track's stated scope and the pattern Btrust-backed judges have rewarded: practical Bitcoin infrastructure that works, built on tools that already exist.

Fit with the event

WHAT THE EVENT ASKS FOR	HOW FIREWATCH MATCHES
Bitcoin/Lightning applications built from scratch, with Lightning Network, LDK and BDK tools in scope DevFest Kaduna	A new Rust tool for Lightning nodes; checks LND, Core Lightning and LDK
Onboarding mid-to-senior developers into Bitcoin open-source work DevFest Kaduna	Open advisory format anyone can contribute to by PR; a Rust codebase other learners can extend
Btrust funded the track to teach running nodes, Lightning development and open-source contribution Btrust event grants	Built for exactly the people that track creates: developers who now run their own nodes
Grants pathway on demo day	A maintained advisory database is the kind of ongoing open-source work grant programs fund

Fit with past winners

This is the first edition of the Build-a-thon, so the closest guides are two hackathons with Btrust support and judges from the same community.

PAST WINNER OR CRITERION	PATTERN	FIREWATCH
bitcoin++ Nairobi 2026: Filter won the open-source spam track; Core-Gate also built maintainer triage recap	Concrete, explainable tooling for a real ecosystem pain, with overrides instead of blind automation	Explainable verdicts, every alert linked to its source, Can't tell instead of a guess, a human makes every change

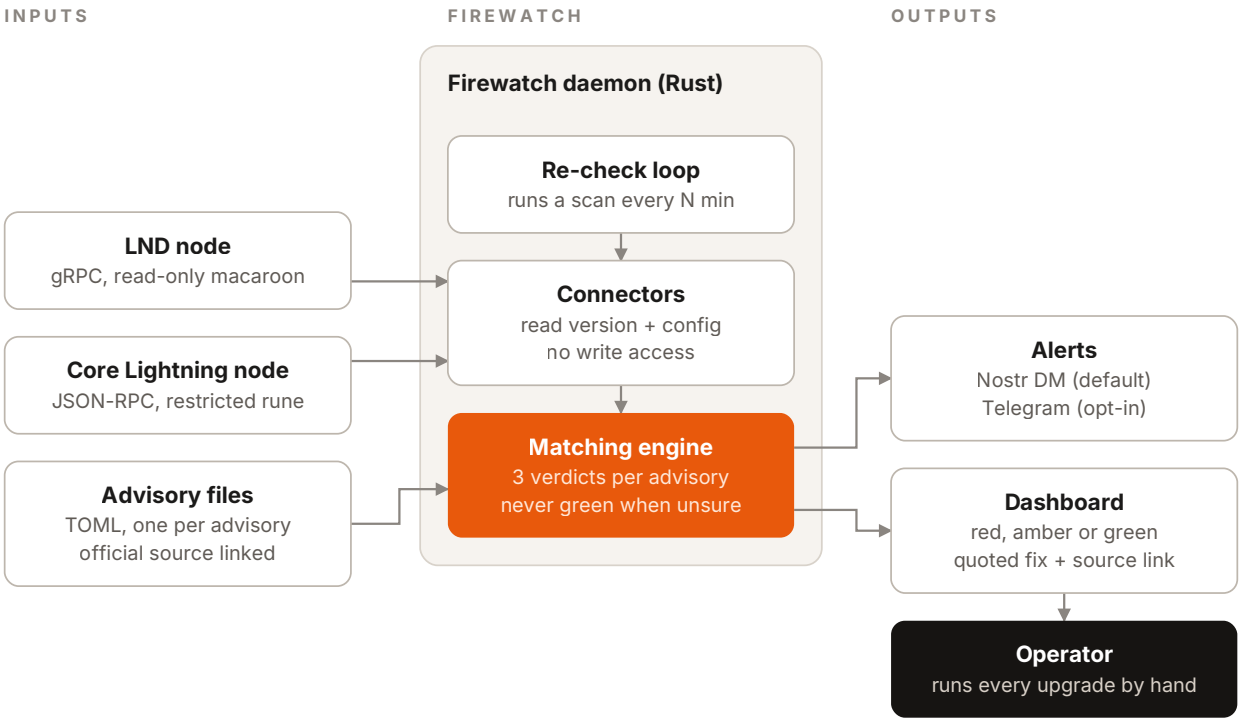
bitcoin++ Nairobi 2026: organizers named pragmatism the strongest through-line recap	Working software over vision, using tools already available	Built on existing RPCs, read-only macaroons and runes, Nostr; demoed live on Polar
bitcoin++ Nairobi 2026: UTXOracle calculator won Best Contribution to Open Source recap	Removing a trusted dependency	Operators stop relying on news sites and X to learn they are exposed
Hack4Freedom Kaduna 2025: judged on originality, technical execution, local relevance and impact Hack4Freedom	Same city, likely overlapping judges	**Originality:** no existing tool matches advisories to node config. **Execution:** tested Rust matching engine. **Impact:** attacks on unpatched nodes in Oct 2026

> **Where it is weaker, and how to pitch around it.** Past overall winners such as WestToEast (mobile money to M-PESA over Lightning) solved everyday payments for ordinary people; Firewatch serves node operators, a smaller group. Pitch it as protecting the operators Kaduna is producing: BitDevs Kaduna, Africa Free Routing bootcamps and this very track are training developers to run nodes, and none of them should learn about an exploit from X after losing funds. Say the audience out loud in the demo; judges respect a clearly scoped user over an inflated one.

04

Architecture

A single Rust daemon reads each node, matches it against the advisory files, and reports. It holds no credential that can change the node.



Credentials are read-only, so the only path from an alert to a change on the node runs through the operator.

CONCERN	PLANNED CRATE
LND gRPC	`tonic`
Core Lightning	a CLN JSON-RPC client
Advisory files	`serde` + `toml`
Alerts	`nostr-sdk`, `teloxide`
Dashboard API	`axum` serving JSON

05

Advisory database

Each advisory is one TOML file, reviewed by a named team member, that always links to the developers' own release notes or statement.

```toml
id = "FW-2026-002"
implementation = "cln"
title = "Multiple fund-loss and crash bugs"
affected = "<= 26.06.7"
fixed = "26.06.8"
severity = "critical"
exploited_in_wild = true
source = "<official Core Lightning release or advisory URL>"

# every condition must hold for 'Affected'
# a condition we cannot read over RPC makes the verdict 'Can't tell'
[[conditions]]
kind = "version_range"

[remediation]
action = "upgrade"   # or "offline" or "disable_feature"
steps = "Quoted from the source, never written by us"
```

Seed list

Only rows marked verified go into the demo; the rest need the official source checked before Oct 29.

ID	SOFTWARE	AFFECTED	FIXED	TRIGGER CONDITION	ACTION	SOURCE
FW-2026-001	LDK	below 0.2.6 (confirm range)	0.2.6	Node initiates a splice	Upgrade	Verified release notes
FW-2026-002	LDK	below 0.2.6 (confirm range)	0.2.6	Node forwards HTLCs; a bogus HTLC reuses a forwarded payment hash	Upgrade	Verified release notes
FW-2026-003	Core Lightning	26.06.7 and earlier	26.06.8	Version only	Upgrade; attacks reported	News only find official notes
FW-2026-004	Core Lightning	26.04 and earlier	none at the time	Version only	Take offline (Aug 26)	News only find official statement
FW-2026-005	LND	before 0.21.0	0.21.0	Unknown	Upgrade	News only details needed

> LDK is a library inside apps, so Firewatch checks it by scanning an app's `Cargo.lock` for the `lightning` crate version, not over a node RPC. RustSec and cargo-audit may already list these, so check for overlap first.

06

Verdict logic and security model

Firewatch never shows green when it is unsure, and it can never change or spend from the node.

VERDICT	WHEN	WHAT THE OPERATOR SEES
Affected	Version in range and every condition confirmed true	Red alert, the source's remediation quoted, source link
Can't tell	Version in range, but a condition cannot be read over RPC	Amber: which setting to check by hand, and how
Not affected	Version outside range, or a condition confirmed false	Green, with the reason shown

Version parsing handles release candidates (`0.21.0-beta.rc1`), CLN's `v26.06.7` style, and packaged builds (Umbrel, Start9). An unparseable version is Can't tell, never Not affected.

What each connector can read

- **Core Lightning:** `getinfo` (version, network) and `listconfigs` (enabled options, including experimental features).
- **LND:** `GetInfo` (version, feature bits). Many `lnd.conf` options are not exposed over RPC, so conditions on them return Can't tell. Verify per advisory before the build day.

Security model

- **Read-only credentials.** LND `readonly.macaroon`; a CLN rune restricted to `getinfo` and `listconfigs`.
- **Advises, never acts.** No auto-upgrade, no auto-shutdown.
- **No invented fixes.** Remediation is quoted from the developers, with the link in every alert.
- **Private alerts by default.** Nostr encrypted DM (NIP-17); Telegram opt-in, since a vulnerability alert on a third-party server tells others which node to attack.
- **Signed advisory database.** Versioned in the repo; signed releases so a tampered feed cannot hide a real advisory.

07

Demo script

One node goes red, one goes amber, and the red one turns green after an upgrade, in under three minutes.

1. **Setup** BEFORE YOU START
   A Polar regtest network with three nodes: an old Core Lightning (26.06.7 or earlier), a current Core Lightning, and an LND node. Firewatch dashboard open, a phone with a Nostr client on the projector.
2. **Hook** 15 S
   "On October 2, Core Lightning warned that attackers were targeting unpatched nodes. Would you have known your node was one of them?"
3. **Scan** 30 S
   Run `firewatch check`. Old CLN turns Affected with FW-2026-003; the phone buzzes with the encrypted alert and the quoted fix.
4. **Honesty moment** 30 S
   The LND node shows Can't tell for an advisory whose condition is not readable over RPC. "We never show green when we are not sure."
5. **Fix** 45 S
   Upgrade the old node in Polar. Firewatch re-checks automatically and it turns Not affected with the reason shown.
6. **Close** 30 S
   Read-only credentials, quoted fixes, an open advisory format anyone can contribute to.

> **Pre-check before Oct 29:** confirm Polar can run the exact vulnerable CLN version. If it cannot, build that version from source in Docker and record a backup video of the full run.

08

Team, build day and prep

Each person owns one component with a clear "done" line, and the must-have demo is finished before any stretch work starts.

OWNER	COMPONENT	DONE MEANS
Rust 1 (Dave)	Matching engine + advisory loader	Loads TOML files, returns the three verdicts, unit test per advisory. Also owns advisory research and defends every condition in Q&A.
Rust 2	Node connectors	Reads version and config from CLN and LND with read-only credentials
Rust 3	Alerts + re-check loop	Nostr DM and Telegram alerts, re-check every N minutes, CLI `firewatch check`

Frontend	Dashboard	Node list with red / amber / green, the reason, the quoted fix and source link

Build day order

Hours counted from the start; exact times once organizers publish them.

HOURS	WORK
0 to 1	Agree shared types (`Advisory` , `NodeInfo` , `Verdict`) and the dashboard JSON
1 to 4	Each owner builds against stub data
4 to 6	Wire together; first full run against Polar
6 to 7	Rehearse the demo twice; fix only what breaks it
Last hour	README with exact test counts and limitations, backup video, push

Must-have: CLN connector, matching engine, Nostr alert, dashboard, the demo run.
Stretch: LND conditions, Telegram, Cargo.lock scan for LDK apps, signed advisory releases.

Prep before Oct 29

No project code; confirm with organizers what else is allowed.

- [ ] Ask GDG Kaduna: is pre-building allowed, judging criteria and prizes, build day start time
- [ ] Find official sources for FW-2026-003, 004, 005 and finish their conditions
- [ ] Check RustSec and cargo-audit for overlap on the LDK advisories
- [ ] Install Polar on every laptop; confirm the vulnerable CLN version runs
- [ ] Test the read-only macaroon and a restricted CLN rune by hand
- [ ] Pick the Rust crates and confirm they build
- [ ] Dashboard wireframe agreed by the team
- [ ] Attend the Oct 21, 22 and 24 masterclasses; ask mentors what they would want from this tool
- [ ] Plan power and internet backup for the venue (power bank, mobile hotspot)

09

Risks, limitations and judge Q&A

The biggest risk is wrong advisory data, so the demo uses only verified advisories and every alert links its source.

Stated limitations (put these in the README as written)

- Firewatch covers only the advisories in its database; it does not discover new vulnerabilities.
- Conditions that depend on `lnd.conf` options not exposed over RPC return Can't tell, not a verdict.
- It serves self-hosted node operators, not users of custodial or LSP-managed phone wallets.
- The advisory database is maintained by people; a missed advisory is a missed alert.

Likely judge questions

QUESTION	ANSWER

How do you know your advisory data is right?	Every entry links the developers' own source, is reviewed by a named team member, and unreadable conditions become Can't tell.
What if Firewatch itself is hacked?	It holds only read-only credentials; it cannot spend, close channels or change settings.
Why not just watch GitHub releases?	A release says a version exists; Firewatch says whether your node and config are affected, and what the developers said to do.
Does cargo-audit already do this?	For LDK in Rust apps it may; Firewatch's core is running LND and CLN nodes, which cargo-audit does not see.
Who maintains the advisories after the hackathon?	An open format in a public repo; node operators and implementation teams can submit entries by PR.
Why does this matter in Kaduna?	Local bootcamps are producing new node operators who do not follow developer channels; they need the warning to reach them.

Sources

- LDK v0.2.6 release notes
- Cointelegraph: Core Lightning urges upgrade amid attacks
- Crypto Briefing: Core Lightning tells operators to go offline
- CryptoSlate: LDK fund-theft and restart bugs
- lightning-sentinel crate
- DevFest Kaduna 2026

A

Registration form answers

Ready to paste. Personal fields (name, email, phone, Lightning address) are left for each member to fill in.

FORM FIELD	ANSWER
Participation Track	**Track 1: Greenfield Build**
Participation Type	**Team (2-4 members)**
Team Name	Firewatch (team members to confirm)
Bitcoin / Lightning Experience Level	Intermediate (Familiar with BDK, LDK, Bitcoin Core, or Nostr)
Primary Programming Languages	Rust, TypeScript / JavaScript
Project / Idea Title	Firewatch: security alerts for Lightning nodes
Target Repositories	N/A (Track 1). Integrates with LND and Core Lightning RPC; advisories sourced from official LND, Core Lightning and LDK releases
Weekly Time Commitment	10-20 hrs/week (adjust to your real availability)

RELEVANT PAST PROJECTS / PORTFOLIOS

Rust for Bitcoin 2.0 cohort (Kaduna): built a Bitcoin block explorer in Rust against a live node. Merged PRs in rust-bitcoin/rust-bitcoin and summa-tx/coins; reviewing in bitcoindevkit/bdk_wallet. GitHub: Chibey-max. Add teammates' links here.

PROJECT OVERVIEW & PROBLEM STATEMENT

Lightning nodes hold real money, and their software gets security fixes often. In August to October 2026 alone, Core Lightning, LND and LDK all shipped security fixes, and Core Lightning reported attackers targeting unpatched nodes. Operators learn late, cannot easily tell whether their own configuration is affected, and do not know the right action when no patch exists yet. Firewatch is a read-only Rust daemon that connects to an LND or Core Lightning node, matches its version and configuration against a curated advisory database, and sends an encrypted Nostr alert (or Telegram) with the developers' own remediation steps, linked to the official source. Each check returns Affected, Not affected, or Can't tell, so the tool never reports safe when it is unsure.

EXPECTED MILESTONES & DELIVERABLES

1. Advisory database: 3 to 5 real 2026 advisories in a documented, machine-readable format with official source links.
2. Node connectors for LND (gRPC, read-only macaroon) and Core Lightning (RPC, restricted rune).
3. Matching engine with three verdicts and unit tests per advisory.
4. Alerts over Nostr encrypted DM and Telegram, plus a re-check after upgrade.
5. Dashboard showing each node's verdicts and the required action.
6. Live demo: a vulnerable node in Polar is flagged, upgraded, and turns safe. Open-source repo with README, test counts and stated limitations.